

CornDrop

Sistema Inteligente de Alimentação para Galinhas

DOCUMENTAÇÃO DE ESCOPO DO PROJETO

Versão 1.0
Abril de 2026

Automação no Agronegócio | IoT | Software + Hardware

1. Identificação do Projeto

Nome do Projeto	CornDrop
Tipo	Sistema IoT — Produto Físico + Aplicativo Digital
Versão do Documento	1.0
Data de Elaboração	Abril de 2026
Status do Projeto	Em Desenvolvimento

2. Visão Geral do Projeto

O CornDrop é um sistema automatizado de alimentação para galinhas, desenvolvido para facilitar a rotina de pequenos produtores rurais. O dispositivo permite programar a liberação de milho de forma automática por meio de um aplicativo, garantindo praticidade, controle e eficiência no manejo alimentar.

2.1 Objetivos do Projeto

Este projeto foi desenvolvido para demonstrar:

- Aplicação de análise de requisitos em um produto físico + digital
- Integração entre software e hardware (IoT)
- Automação de processos no agronegócio
- Desenvolvimento de software
- Uso de microcontroladores em soluções reais de mercado

2.2 Contexto do Problema

Pequenos criadores de galinhas enfrentam desafios recorrentes no dia a dia do manejo alimentar:

- Alimentação manual em horários fixos
- Falta de controle preciso sobre a quantidade de ração fornecida
- Dificuldade em monitorar remotamente a quantidade de alimento

O CornDrop resolve esses problemas automatizando o processo de alimentação e permitindo controle e monitoramento remoto por meio de uma aplicação web.

3. Público-Alvo

Segmento	Perfil	Necessidade Principal
Pequenos Produtores Rurais	Produtores com ate 500 aves	Redução de custo operacional
Criadores Domésticos	Famílias com galinhas no quintal	Praticidade no dia a dia
Agricultores Familiares	Rotina manual intensa	Automação e controle remoto

4. Escopo do Projeto

4.1 Dentro do Escopo

- Configuração de horários automáticos de alimentação via aplicativo
- Liberação automática e controlada de milho por servomotor
- Monitoramento em tempo real do nível de alimento no reservatório
- Notificações ao usuário quando o nível de milho estiver baixo
- Execução das rotinas programadas de forma autônoma
- Interface web para controle e visualização do dispositivo
- Comunicação entre aplicativo e dispositivo via Wi-Fi

4.2 Fora do Escopo (nesta versão)

- Suporte a múltiplos dispositivos por conta
- Analise histórica de consumo de ração com gráficos simples
- Versão mobile do painel de controle

5. Funcionalidades Principais

Funcionalidade	Descrição
Programação de Horários	O usuário define horários de alimentação pela aplicação
Liberação Automática de Milho	Servomotor aciona a abertura do reservatório nos horários configurados
Controle Remoto	O usuário pode acionar a alimentação manualmente pela interface da aplicação
Notificações de Quantidade de Insumo	Alerta automático quando o milho estiver acabando
Execução Autônoma	O dispositivo executa rotinas sem necessidade de conexão ativa

6. Arquitetura do Sistema

6.1 Back-end

- Linguagem/Framework: Java + Spring Boot
- Banco de Dados: PostgreSQL
- Containerização: Docker
- Orquestração: Docker Compose
- Comunicação com dispositivo: Wi-Fi / Serial

6.2 Front-end

- React

6.3 Hardware

- Microcontrolador: ESP32
- Atuador: Servomotor para liberação de milho
- Estrutura física do reservatório.

7. Pré-requisitos Técnicos

Para executar o projeto localmente, os seguintes softwares devem estar instalados:

Tecnologia	Versão Mínima	Finalidade
Docker	Ultima estável	Containerização dos serviços
Java JDK	17	Execução do back-end
Docker Compose	Ultima estável	Orquestração dos contêineres

8. Critérios de Aceitação

O projeto será considerado concluído e aceito quando todos os critérios abaixo forem satisfeitos:

- O sistema deve realizar a liberação automática de milho nos horários configurados pelo usuário.
- A liberação de milho só deve ocorrer caso haja estoque disponível no dispositivo.
- O sistema deve seguir corretamente a rotina de alimentação definida pelo usuário.
- O usuário deve conseguir visualizar e acompanhar a quantidade de milho pelo aplicativo.
- O sistema deve notificar o usuário quando a quantidade de milho estiver baixo.

- O dispositivo deve continuar executando as rotinas programadas mesmo sem interação ativa do usuário.

9. Backlog

O backlog completo do projeto, com épicos e histórias de usuário detalhadas, está estruturado e disponível no Projeto do GitHub e no documento de backlog.

Epic	Descrição
EP-01 — Autenticação e gestão de conta	Cobre todo o ciclo de identidade do usuário na plataforma: cadastro, login com JWT, recuperação de senha e edição de perfil. É a base que desbloqueia o acesso a todas as demais funcionalidades. Sem esse épico nenhuma outra tela ou recurso pode ser utilizado.
EP-02 — Vinculação e configuração do dispositivo	Trata da ponte entre a conta do usuário e o hardware físico. Inclui o fluxo de pareamento via código único gerado pelo ESP32, monitoramento do status de conexão em tempo real, personalização do nome do dispositivo e desvinculação. É o épico de maior risco técnico por envolver a integração direta entre software e hardware IoT.
EP-03 — Programação de rotinas de alimentação	Entrega o núcleo do produto: a capacidade de criar, editar, pausar e excluir horários automáticos de liberação de milho. Inclui a sincronização das rotinas com a memória não-volátil do ESP32, garantindo que o dispositivo continue executando os horários programados mesmo sem internet ativa — o diferencial técnico central do CornDrop.
EP-04 — Controle manual e monitoramento em tempo real	Oferece ao produtor visibilidade e controle imediato do sistema. Cobre o acionamento manual de liberação pelo aplicativo, a leitura do sensor de nível do reservatório, o histórico completo de alimentações automáticas e manuais e o dashboard de resumo diário. É o épico de maior valor percebido pelo usuário final no dia a dia.
EP-05 — Notificações e alertas	Garante que o produtor seja informado proativamente sobre eventos críticos sem precisar abrir o aplicativo. Cobre o alerta de estoque baixo (nível $\leq 20\%$), a notificação de dispositivo offline por mais de 1 hora e a central de preferências onde o usuário escolhe quais alertas receber e por qual canal (e-mail ou push).
EP-06 — Infraestrutura, DevOps e hardware	Épico técnico que sustenta todos os outros. Cobre a containerização do back-end Spring Boot com PostgreSQL via Docker Compose, a comunicação segura entre ESP32 e API via HTTPS com autenticação por token, e a integração do sensor de nível com envio periódico de leituras à API. Precisa estar concluído antes que qualquer funcionalidade de produto possa ser testada de ponta a ponta.
EP-07 — Desenvolvimento de telas	Implementa as interfaces visuais baseadas nos protótipos de alta fidelidade. Cobre as quatro telas principais (Dashboard, Devices, Stock e Schedule) com todos os seus componentes, cards, gráficos, formulários e listas, além da barra de navegação superior para desktop e a bottom navigation para mobile. Cada história deste épico está diretamente rastreável a um elemento específico dos protótipos entregues.

10. Estrutura da Documentação Inicial

A documentação completa do projeto esta distribuída conforme abaixo:

Documento	Localização
Repositório	Github
Home	Home
Configuração do Ambiente	Configurar Ambiente
Banco de Dados	Banco de Dados
Backlog (User Stories)	Backlog

11. Restrições e Premissas

11.1 Restrições

- O sistema suporta apenas um dispositivo por instância na versão atual.
- A comunicação depende de rede Wi-Fi disponível no local de instalação.

11.2 Premissas

- O usuário possui smartphone com acesso à internet
- O local de instalação possui rede Wi-Fi estável
- O reservatório será reabastecido manualmente pelo usuário
- A estrutura física do reservatório será construída pelos membros do projeto.

12. Histórico de Versões

Versão	Data	Descrição	Autor
1.0	Abr/2026	Versão inicial da documentação de escopo	Cauane Oliveira
1.1	May/2026	Ajuste de Backlog	Cauane Oliveira